

Guion de la presentación

AI Media Generator · Spec-Driven Development con Spec Kit · 10 minutos

Todo se construye antes. Un agente tarda 5–15 min por ticket; en 10 minutos no cabe. En vivo se lanzan los comandos, se habla mientras corren, y se demuestra la app ya terminada.

Un `npm test` antes de cada push. Una línea suelta, sin encadenar con nada. El agente ya corre las pruebas mientras construye, así que esta pasa en verde — no es para descubrir errores, es para verlas pasar antes de subir y poder decirlo en voz alta. Si sale en rojo, no subas: es que el agente dejó algo a medias.

Dos carpetas, no una. El **repo real** queda con todo construido y el `.env.local` puesto — de ahí sale el `npm run dev` del final. La **carpeta de demo** queda en el estado justo después de la fase 1 — ahí se lanzan los comandos en vivo y los agentes sí arrancan de verdad.

Antes de la presentación

Todos estos comandos se corren antes, en el orden en que están. Sin esto no hay app que mostrar.

1 SERGIO · LA FASE 1 EN MAIN — YA ESTÁ HECHO

T001–T009 contruidos, 72 pruebas en verde, lint y build pasando, subido a `main` y marcado con dos tags: `fase-1` (la base construida, de ahí sale la carpeta de demo) y `pre-fase-1` (el repo sin código, para practicar). **No hay que volver a correrlo.** Esto solo comprueba que sigue en pie.

```
git fetch origin --tags
git log --oneline -1 origin/main
git tag -l

# Tiene que decir:
# a848af0 README: la fase 1 ya esta en main...
# fase-1
# pre-fase-1
```

Los tres pueden arrancar desde ya.

2 MATEO · JOHAN · TOMÁS · CONSTRUYEN SUS FASES

Los tres al mismo tiempo, cada uno en su máquina. Cada bloque completo, de arriba abajo.

```
# — MATEO — las pantallas
git checkout main && git pull origin main
git checkout -b 001-frontend
/speckit-implement fase 2
npm test
git add -A && git commit -m "Fase 2 - pantallas"
git push -u origin 001-frontend
```

```
# — JOHAN — el backend
git checkout main && git pull origin main
git checkout -b 001-backend
/speckit-implement fase 3
npm test
git add -A && git commit -m "Fase 3 - backend"
git push -u origin 001-backend
```

```
# — TOMÁS — el dashboard
git checkout main && git pull origin main
git checkout -b 001-dashboard
/speckit-implement fase 4
npm test
git add -A && git commit -m "Fase 4 - dashboard"
git push -u origin 001-dashboard
```

3 TOMÁS · LAS CUENTAS REALES (T030)

Este ticket es el único con trabajo a mano: crear las cuentas y copiar las llaves. Sin esto el `npm run dev` del final no genera nada. Va dentro de la fase 4, pero el agente no puede hacerlo solo.

```
cp .env.local.example .env.local
# Llena las ocho llaves a mano y pásale el archivo a Santiago.
# Clerk · MongoDB Atlas · Vercel Blob · Higgsfield
# .env.local NUNCA se sube: está en .gitignore
```

4 SANTIAGO · JUNTA LAS TRES RAMAS Y CIERRA

Dos cosas rompen aquí. **Uno por uno:** un merge de tres ramas a la vez se cancela entero si encuentra un conflicto, y el T031 dice que `package.json` va a conflictuar. Con `origin/`: Santiago no tiene esas ramas en local, solo `main` — por eso el `fetch` primero.

```
git checkout main && git pull origin main
git fetch origin

# Uno por uno. Nunca los tres en un solo comando.
git merge origin/001-frontend
git merge origin/001-backend
git merge origin/001-dashboard

/speckit-implement fase 5

npm test
git add -A && git commit -m "Fase 5 - integracion"
git push origin main
```

Copia el `.env.local` de Tomás a la raíz del repo real y comprueba que `npm run dev` genera una imagen de verdad. Este es el repo del que sale la demo del final, y es de Santiago.

5 LOS CINCO · LA CARPETA DE DEMO

Cada uno en su máquina, en una carpeta **aparte** del repo real. Queda en el estado de la fase 1, así que los comandos que se lanzan en vivo arrancan de verdad y las ramas no chocan con las que ya están subidas.

```
git clone https://github.com/sjunka/speckit-ai-generator.git demo-speckit
cd demo-speckit
git checkout fase-1
npm install

# Comprueba que el agente encuentra sus archivos
bash .specify/scripts/bash/check-prerequisites.sh --json --require-tasks --include-tasks
```

Tiene que imprimir una línea con `FEATURE_DIR`. Abre ahí tu agente — Claude Code, Cursor, Copilot, el que uses — y deja la carpeta lista en pantalla.

6 CINCO MINUTOS ANTES

Quién	Ventana 1 — demo-speckit	Ventana 2 — repo real
Sergio	Tu agente abierto, y <code>spec.md</code> , <code>plan.md</code> y <code>tasks.md</code> ya abiertos en pestañas	—
Mateo	Tu agente abierto en la carpeta	—
Johan	Tu agente abierto en la carpeta	—
Tomás	Tu agente abierto en la carpeta	El celular con la app y <code>/dashboard</code> abierto
Santiago	Tu agente abierto en la carpeta	<code>npm run dev</code> corriendo y el navegador en <code>localhost:3000</code>

Los 10 minutos

De aquí en adelante, todo lo que se corre en vivo va en la carpeta `demo-speckit`, salvo el `npm run dev` del minuto 6:15.

Min	Quién	Qué pasa	Comando
0:00	Sergio	Abre: qué es SDD	—
0:45	Sergio	Muestra las specs	—
1:45	Los tres	Lanzan sus comandos a la vez	<code>git checkout -b · /speckit-implement</code>
2:45	Sergio	Explica por qué no chocan	—
4:15	Mateo · Johan · Tomás	20 s cada uno	—
5:15	Santiago	El merge y el T032	<code>git merge · /speckit-implement fase 5</code>
6:15	Todos	La app funcionando	<code>npm run dev</code>
8:45	Sergio	Cierre	—

0:00

SERGIO

45 s

DICE

«Buenas noches. Vamos a construir una aplicación completa sin escribir una línea de código a mano.» «Se llama Spec-Driven Development. En vez de escribir código, escribimos la especificación. El agente la lee y construye.» «La especificación es la fuente de verdad. El código es la salida.»

0:45

SERGIO

1 min

Abre el repo en pantalla. Muestra `specs/001-ai-media-generator/` con los tres archivos.

```
ls specs/001-ai-media-generator/  
# spec.md  plan.md  tasks.md
```

DICE

«Aquí está todo lo que el agente necesita. Tres archivos.» «`spec.md` dice qué hace la app, sin nada de tecnología. `plan.md` dice cómo se construye: el stack y los contratos. Y `tasks.md` tiene 39 tickets ordenados.» «Ninguno tiene código. Son descripciones. El agente hace el resto.»

SERGIO DICE

«Repartimos el trabajo en cinco fases. La primera ya está: crea el proyecto y las piezas compartidas. Ahora Mateo, Johan y Tomás arrancan las suyas al mismo tiempo.»

Los tres lanzan, en su propia pantalla, dentro de `demo-speckit`, a la vez:

```
# — MATEO
git checkout -b 001-frontend
/speckit-implement fase 2

# — JOHAN
git checkout -b 001-backend
/speckit-implement fase 3

# — TOMÁS
git checkout -b 001-dashboard
/speckit-implement fase 4
```

Déjenlos corriendo. No esperen a que terminen, y no los cierren: se ven trabajando en el fondo durante los siguientes cuatro minutos.

Los dos errores que rompen esto: es `/speckit-implement`, no `/implement`; y nunca sin decir la fase — sin argumento el agente construye las cinco.

2:45 SERGIO

1 min 30 s

Mientras los agentes trabajan, abre `tasks.md` y señala dos fases distintas.

DICE

«Tres personas escribiendo en el mismo proyecto al mismo tiempo. Normalmente eso termina en conflictos. Aquí no, y esta es la razón.» «Cada fase declara de qué archivos es dueña, en el bloque `Owns`, y cuáles no toca nunca, en `Never touches`.» «Lo que Mateo posee está en la lista de "nunca toca" de Johan. Y al revés. Ningún archivo aparece en dos listas de dueños, así que dos personas nunca editan lo mismo.» «Y cuando una fase necesita algo de otra que todavía no existe, la usa por la firma acordada en `plan.md` y la reemplaza por un doble. Por eso nadie espera a nadie.»

4:15 MATEO · JOHAN · TOMÁS

20 s cada uno

MATEO

«Yo construyo las pantallas: la portada, el inicio de sesión, tomar la foto, y la pantalla del video. Todas hablan con el backend por HTTP, nunca directo.»

JOHAN

«Yo construyo el backend: la conexión con la IA que genera la imagen y el video, el almacenamiento, y las rutas. Mis pruebas corren sin llaves y sin internet.»

TOMÁS

«Yo construyo el panel del dueño: un interruptor que apaga toda la generación, la calidad del video, los contadores y el gasto estimado.»

DICE

«Cuando las tres ramas terminan, cada uno corre las pruebas, ve que están en verde, y sube la suya. Yo las junto. Esa es la fase 5.»

```
# Cada uno corre sus pruebas y sube la suya
npm test
git push -u origin 001-frontend
git push -u origin 001-backend
git push -u origin 001-dashboard

# Santiago las junta, una por una
git checkout main && git fetch origin
git merge origin/001-frontend
git merge origin/001-backend
git merge origin/001-dashboard

/speckit-implement fase 5
```

«Y ahí está el ticket más importante de todos, el T032.» «Mateo probó sus pantallas contra un backend falso, no contra el de Johan. Sus pruebas pasan. Las de Johan también. Y aun así la app podría estar rota, porque nadie comprobó que los dos se comporten igual.» «El T032 es exactamente esa comprobación. Tests verdes no significan app funcionando.»

Cambia a la ventana del repo real, el que ya tiene las cinco fases y el `.env.local` puesto. Aquí no se corre nada de la carpeta de demo.

```
npm run dev
# http://localhost:3000
```

SANTIAGO DICE

«Esta es la aplicación, corriendo con cuentas reales.»

El recorrido, en el celular: iniciar sesión → tomar una foto → elegir el estado de ánimo → generar la imagen → convertirla en video → descargarlo.

TOMÁS MUESTRA EL PANEL

«Y desde aquí el dueño apaga toda la generación con un interruptor, sin volver a desplegar.»

Abre `localhost:3000/dashboard`. Apaga el interruptor, intenta generar, muestra el mensaje de pausa. Vuelve a encenderlo.

DICE

«Resumiendo: escribimos una especificación, no código.» «De ahí salieron 39 tickets que cinco personas ejecutamos en paralelo sin pisarnos, porque cada fase declara de qué archivos es dueña.» «El resultado es esta aplicación funcionando. Gracias.»

Todos los comandos, en orden

La lista completa de una sola mirada. Si algo se pierde en la presentación, esta es la página a la que volver.

#	Cuándo	Quién	Comando
1	Hecho	Sergio	git checkout main
2	Hecho	Sergio	git merge build-verificado
3	Hecho	Sergio	git push origin main
4	Hecho	Sergio	git tag pre-fase-1 && git tag fase-1
5	Hecho	Sergio	git push origin main pre-fase-1 fase-1
6	Antes	Mateo	git checkout main && git pull origin main
7	Antes	Mateo	git checkout -b 001-frontend
8	Antes	Mateo	/speckit-implement fase 2
9	Antes	Mateo	npm test
10	Antes	Mateo	git add -A && git commit -m "Fase 2 - pantallas"
11	Antes	Mateo	git push -u origin 001-frontend
12	Antes	Johan	git checkout main && git pull origin main
13	Antes	Johan	git checkout -b 001-backend
14	Antes	Johan	/speckit-implement fase 3
15	Antes	Johan	npm test
16	Antes	Johan	git add -A && git commit -m "Fase 3 - backend"
17	Antes	Johan	git push -u origin 001-backend
18	Antes	Tomás	git checkout main && git pull origin main
19	Antes	Tomás	git checkout -b 001-dashboard
20	Antes	Tomás	/speckit-implement fase 4
21	Antes	Tomás	cp .env.local.example .env.local ← a mano, y pásaselo a Santiago
22	Antes	Tomás	npm test
23	Antes	Tomás	git add -A && git commit -m "Fase 4 - dashboard"
24	Antes	Tomás	git push -u origin 001-dashboard
25	Antes	Santiago	git checkout main && git pull origin main
26	Antes	Santiago	git fetch origin
27	Antes	Santiago	git merge origin/001-frontend
28	Antes	Santiago	git merge origin/001-backend
29	Antes	Santiago	git merge origin/001-dashboard
30	Antes	Santiago	/speckit-implement fase 5
31	Antes	Santiago	npm test

32	Antes	Santiago	<code>git add -A && git commit -m "Fase 5 - integracion"</code>
33	Antes	Santiago	<code>git push origin main</code>
34	Antes	Los cinco	<code>git clone https://github.com/sjunka/speckit-ai-generator.git demo-speckit</code>
35	Antes	Los cinco	<code>cd demo-speckit && git checkout fase-1</code>
36	Antes	Los cinco	<code>npm install</code>
37	Antes	Los cinco	<code>bash .specify/scripts/bash/check-prerequisites.sh --json --require-tasks --include-tasks</code>
38	0:45	Sergio	<code>ls specs/001-ai-media-generator/</code>
39	1:45	Mateo	<code>git checkout -b 001-frontend</code>
40	1:45	Mateo	<code>/speckit-implement fase 2</code>
41	1:45	Johan	<code>git checkout -b 001-backend</code>
42	1:45	Johan	<code>/speckit-implement fase 3</code>
43	1:45	Tomás	<code>git checkout -b 001-dashboard</code>
44	1:45	Tomás	<code>/speckit-implement fase 4</code>
45	5:15	Los tres	<code>git push -u origin 001-frontend / -backend / -dashboard</code>
46	5:15	Santiago	<code>git checkout main && git fetch origin</code>
47	5:15	Santiago	<code>git merge origin/001-frontend</code>
48	5:15	Santiago	<code>git merge origin/001-backend</code>
49	5:15	Santiago	<code>git merge origin/001-dashboard</code>
50	5:15	Santiago	<code>/speckit-implement fase 5</code>
51	6:15	Santiago	<code>npm run dev</code> ← en el repo real

SI ALGO SE CAE EN VIVO

Pasa esto	Haz esto
<code>fatal: a branch named '001-frontend' already exists</code>	Estás en el repo real, no en <code>demo-speckit</code> . Cambia de carpeta.
El agente no encuentra <code>tasks.md</code>	Corre el <code>check-prerequisites.sh</code> . Si no imprime <code>FEATURE_DIR</code> , estás fuera del repo.
El agente empieza a construir las cinco fases	Se te olvidó el argumento. Párralo y corre <code>/speckit-implement fase N</code> .
La app no genera nada en el minuto 6:15	Falta el <code>.env.local</code> en la raíz del repo real. Cópialo y reinicia <code>npm run dev</code> .
Un merge queda en conflicto	Es <code>package.json</code> , y es lo esperado: deja las dependencias de las dos ramas. No lo hagas en vivo.
<code>npm test</code> sale en rojo en la carpeta de demo	Normal: ahí el agente sigue construyendo a media fase. El <code>npm test</code> del guion es el de antes , sobre la fase terminada. En vivo no lo corras.

SI PREGUNTAN

Preguntan	Respondes
¿Qué le escribieron al agente para que saliera ese <code>spec.md</code> ?	Los cinco prompts, uno por comando, están en <code>docs/prompts-specs.md</code> . Ábrelo y muestra el de <code>/speckit.specify</code> .
¿Esto solo funciona con Claude Code?	No. El comando es un archivo de texto del repo, <code>.claude/skills/speckit-implement/SKILL.md</code> , y cualquier agente puede seguirlo. El README tiene la sección Si no tienes Claude Code con el prompt para pegar.
¿Y las pruebas las escribió el agente después del código?	No: la constitución es test-first y el orden se ve en el historial de git. Por eso el <code>npm test</code> antes de subir sale en verde y no descubre nada.